

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Методы трансляции

ОТЧЕТ
к лабораторной работе №1
на тему

**ОПРЕДЕЛЕНИЕ МОДЕЛИ ЯЗЫКА. ВЫБОР ИНСТРУМЕНТАЛЬНОЙ
ЯЗЫКОВОЙ СРЕДЫ**

Студент
Преподаватель

П. Н. Носкович
Н. Ю. Гриценко

Минск 2025

СОДЕРЖАНИЕ

Содержание	2
1 Цель работы	3
2 Подмножество языка программирования	4
2.1 Типы данных в языке программирования <i>PHP</i>	4
2.2 Операторы циклов и условные операторы.....	4
2.3 Структуры	6
3 Инструментальная языковая среда	7
Заключение.....	8
Список использованных источников.....	9
Приложение А (обязательное) Текст программ	10

1 ЦЕЛЬ РАБОТЫ

Цель данной работы заключается в формировании подмножества языка программирования, включая все типы констант, переменных, операторов и функций. В рамках этого подмножества должны быть представлены все доступные типы данных, а также учтены все структуры, доступные в рассматриваемом языке. Также необходимо рассмотреть возможности подключения библиотек (модулей) и других ресурсов.

В работе будут проанализированы операторы циклов, такие как *do...while* и *for*, которые предоставляет язык, а также условные операторы, включая *if...else* и *case*.

Кроме того, будет определена инструментальная языковая среда, включающая язык программирования с указанием версии, операционную систему (например, *Windows* или *Linux*), на которой будет происходить разработка, а также тип компьютера (*PC* или *Macintosh*).

В отчете по лабораторной работе будет представлено полное описание подмножества языка программирования, а также тексты 3 программ, которые включают все элементы этого подмножества. Также будет приведено детальное описание инструментальной языковой среды.

2 ПОДМНОЖЕСТВО ЯЗЫКА ПРОГРАММИРОВАНИЯ

В языке программирования *PHP* существует множество элементов, которые позволяют разрабатывать различные типы программ. В этом разделе мы подробно рассмотрим подмножество языка, включающее типы данных, переменные, операторы циклов, условные операторы и структуры данных, с конкретными примерами.

2.1 Типы данных в языке программирования *PHP*

PHP является языком с динамической типизацией. Это значит, что тип данных переменной выводится во время выполнения, и в отличие от ряда других языков программирования в *PHP* не надо указывать перед переменной тип данных. В *PHP* есть несколько базовых типов данных.

1 Скалярный тип *int* представляет целое число со знаком. Переменная типа *int* занимает в памяти 32 бита, то есть может принимать значения от -2147483648 до 2147483647. Если переменная получает числовое значение вне этого диапазона, то она трактуется как переменная типа *float*.

2 Тип *float* представляет числа с плавающей точкой. Размер числа с плавающей точкой зависит от платформы. Максимально возможное значение, как правило, составляет 1.8E+308 с точностью около 14 десятичных цифр.

3 Тип *string* представляет строки. Для работы с текстом можно применять строки. Строки бывают двух типов: в двойных кавычках и одинарных. От типа кавычек зависит обработка строк интерпретатором. Так, переменные в двойных кавычках заменяются значениями, а переменные в одинарных кавычках остаются неизменными.

4 Логический тип *bool* представляет два значения: *true* (истина) или *false* (ложь). Используется для логических операций.

5 Любой тип *mixed* представляет значение, которое может быть любого типа. Этот тип часто используется в сигнатурах функций, когда возможен любой тип данных.

2.2 Операторы циклов и условные операторы

Рассмотрим условные операторы языка.

1 Оператор *if*. Оператор *if* используется для выполнения блока кода, если заданное условие истинно. Синтаксис представлен на рисунке 1.

```

1  $a = 5;
2  if($a>0){
3      echo "Переменная а больше нуля";
4  }
5  elseif($a < 0){
6      echo "Переменная а меньше нуля";
7  }
8  else{
9      echo "Переменная а равна нулю";
10 }

```

Рисунок 1 – Синтаксис оператора *if*

2 Оператор *SELECT CASE*. Оператор *SELECT CASE* позволяет выбрать один из нескольких блоков кода на основе значения выражения. Синтаксис представлен на рисунке 2.

```

SELECT CASE (score)
  CASE (90:100)
    PRINT *, "Отлично"
  CASE (75:89)
    PRINT *, "Хорошо"
  CASE (50:74)
    PRINT *, "Удовлетворительно"
  CASE DEFAULT
    PRINT *, "Неудовлетворительно"
END SELECT

```

Рисунок 2 – Синтаксис оператора *SELECT CASE*

3 В языке программирования *Fortran* существуют операторы сравнения, такие как == (равно), != (не равно), > (больше), < (меньше), >= (больше или равно), <= (меньше или равно), которые позволяют сравнивать значения различных типов данных. Эти операторы возвращают логическое значение (.TRUE. или .FALSE.) в зависимости от результата сравнения.

4 Операторы логического выражения. В *Fortran* доступны логические операторы .AND. (логическое И), .OR. (логическое ИЛИ), .NOT. (логическое НЕ). Они используются для комбинирования и инвертирования логических значений.

В языке программирования *Fortran* доступны следующие операторы цикла:

1 Оператор цикла *DO*. Оператор *DO* в *Fortran* используется для повторения блока кода определенное количество раз или до выполнения заданного условия. Синтаксис оператора можно увидеть на рисунке 3.

```
DO i = 1, 10  
  sum = sum + i  
END DO
```

Рисунок 3 – Синтаксис оператора *DO*

2 Оператор цикла *DO WHILE*. используется для создания цикла, который повторяет выполнение блока кода, пока заданное условие истинно. Синтаксис можно увидеть на рисунке 4.

```

DO WHILE (number >= 0)
    sum = sum + number
    PRINT *, "Текущая сумма:", sum
    PRINT *, "Введите следующее число"
    READ *, number
END DO

```

Рисунок 4 – Синтаксис оператора *DO WHILE*

3 Операторы управления циклом. Оператор *EXIT* используется для немедленного выхода из цикла. Оператор *CYCLE* используется для пропуска текущей итерации цикла и перехода к следующей итерации. Пример использования операторов *CYCLE* и *EXIT* можно увидеть на рисунке 5.

```

DO i = 1, 10
    IF (i == 5) THEN
        PRINT *, "Выход из цикла на i =", i
        EXIT
    END IF

    IF (MOD(i, 2) == 0) THEN
        PRINT *, "Пропуск четного числа i =", i
        CYCLE
    END IF

    PRINT *, "Нечетное число i =", i
END DO

```

Рисунок 5 – Синтаксис операторов управления циклами

2.3 Структуры

В *Fortran* реализованы пользовательские структуры [1]. Структуры (или пользовательские типы данных) позволяют объединять различные типы данных в единую единицу. Это особенно полезно, когда необходимо хранить связанные данные, которые могут иметь разные типы. Пользовательские структуры позволяют разработчикам создавать более сложные и организованные модели данных. Пример объявления структуры представлен на рисунке 6.

```
TYPE :: Person
  CHARACTER(LEN=20) :: name
  INTEGER :: age
END TYPE Person
```

Рисунок 6 – Пример структуры в языке *Fortran*

Массивы в *Fortran* – это структуры данных, которые позволяют хранить наборы значений одного и того же типа в виде единой переменной. Это удобно для работы с большими объемами данных и упрощает код, так как позволяет обращаться к элементам массива по индексам. Индексация массивов в *Fortran* начинается с 1, то есть первый элемент массива имеет индекс 1. Пример использования массивов представлен на рисунке 7.

```
REAL :: array(10)
INTEGER :: matrix(5, 5)
REAL :: array(5) = [1.0, 2.0, 3.0, 4.0, 5.0]
```

Рисунок 7 – Пример массива в языке *Fortran*

В *Fortran* существуют функции, которые позволяют организовывать код, делая его более структурированным и переиспользуемым. Функции в *Fortran* могут принимать аргументы, выполнять вычисления и возвращать результаты. Функции могут принимать как входные, так и выходные аргументы. Для входных аргументов используется спецификатор *INTENT(IN)*, для выходных – *INTENT(OUT)*.

3 ИНСТРУМЕНТАЛЬНАЯ ЯЗЫКОВАЯ СРЕДА

В качестве языковой среды выбран язык программирования *Go* (версия 1.23.5). Разработка основана на работе с операционной системой *Ubuntu* на ПК.

Инструментальная среда *Go* представляет собой интегрированную среду разработки, которая обеспечивает программистам возможности для создания, отладки и тестирования программ на языке *Go*. Рассмотрим возможности языковой среды.

Инструментальная среда *Go* включает в себя редактор кода, такой как *Visual Studio Code* или *GoLand*, которые предоставляют различные функции, включая подсветку синтаксиса, автодополнение кода, инструменты отладки и рефакторинга.

Для разработки программ на *Go* необходим компилятор, который может преобразовывать исходный код на *Go* в исполняемый файл. Встроенный компилятор *Go* позволяет создавать исполняемые файлы для различных платформ и архитектур, что упрощает кросс-компиляцию.

Инструментальная среда *Go* также включает в себя средства для тестирования, которые позволяют разработчикам писать и запускать тесты для проверки функциональности кода. Это особенно важно для обеспечения качества и надежности программного обеспечения.

Кроме того, *Go* имеет встроенную поддержку управления зависимостями, которая упрощает процесс работы с внешними библиотеками и пакетами.

Инструментальная среда *Go* может иметь интеграцию с системами контроля версий, такими как *Git*. Это позволяет программистам управлять версиями исходного кода, отслеживать изменения и сотрудничать с другими разработчиками.

В данной главе была рассмотрена основная языковая среда для языка *Go*, а также была выбрана платформа, на которой будет проводиться разработка и анализ.

ЗАКЛЮЧЕНИЕ

В ходе работы было определено подмножество языка программирования *Fortran*, включая типы, константы, переменные, операторы и функции. В подмножество языка были включены все основные типы данных, такие как *INTEGER*, *REAL*, *CHARACTER* и *LOGICAL*, операторы цикла и условные операторы. Учтены структурные типы, включая пользовательские структуры, что позволяет группировать связанные данные. Определена инструментальная языковая среда, т. е. язык программирования и операционная система для разработки. В ходе лабораторной работы дается полное определение подмножества языка программирования, тексты трёх программ, включающих все элементы этого подмножества. Приводится подробное описание инструментальной языковой среды.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Fortran High-performance parallel programming language [Электронный ресурс]. – Режим доступа: <https://fortran-lang.org/>. – Дата доступа: 23.01.2025.

ПРИЛОЖЕНИЕ А

(обязательное)

Текст программ

Листинг 1 – module1.f90

```
<?php
$name = "Ivan_Ivanov";
$age = 30;
$height = 1.85;
$array = [5, 10, 15, 25];
$value = 7;

echo "Name:_$name \n";
echo "Age:_$age \n";
echo "Height:_$height \n";

// Сумма элементов массива
$sum = array_sum($array);
echo "Sum_of_array_elements:_$sum \n";

// Проверка диапазона
if ($value >= 6 && $value <= 10) {
    echo "Value_is_in_the_range_6-10 \n";
}

$i = 1;
// Простой DO loop (эмулируется через for в PHP)
for ($i = 1; $i <= 5; $i++) {
    echo "DO_loop_iteration:_$i \n";
}

// DO WHILE loop
$i = 1;
do {
    echo "DO_WHILE_loop_iteration:_$i \n";
    $i++;
} while ($i <= 3);

// Цикл с EXIT и CYCLE (используем break и continue)
for ($i = 1; $i <= 10; $i++) {
    if ($i == 8) {
        break; // EXIT
    }
    if ($i < 3) {
        continue; // CYCLE
    }
    echo "Iteration_with_EXIT_and_CYCLE:_$i \n";
}
?>
```